Inference

Lecture 4: what it costs to run a language model

Aarne Talman

10 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	27 / 29 Oct	Language models	Pretraining	Aarne
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	**10 / 12 Nov**	**Inference**	**Lab 3: serve**	**Aarne**
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. Inference basics *15 min*
2. Decoding *10 min*
3. Prefill vs decode *15 min*
4. The KV cache *15 min*
5. Batching and the server *10 min*
6. Quantisation *10 min*
7. Speculative and constrained decoding *10 min*
8. Cost of serving *5 min*

Take away

By the end of today you should be able to explain where the time and memory go when a model answers, estimate how many users one GPU can serve, and compare two server settings fairly on speed and quality.

Inference basics

Training and inference

Training

The model sees many examples and its **weights** (its billions of numbers) are adjusted step by step. This happens once, or a few times, and takes days to months on many GPUs.

Inference

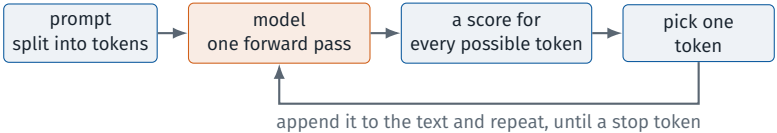
The weights are **fixed**. The model is used to answer prompts: every chat message, every API call, every evaluation run. This happens millions of times, for as long as the model is in use.

Over a model's life, most of the computing goes into inference, not training. That is why it matters how fast and how cheaply a model can answer.

Take away

Today's questions: where do the time and the memory go when a model answers, how many users can one GPU serve, and how can we make it cheaper without making the answers worse?

How a model writes an answer



- A **token** is a word or a piece of a word. Text goes in and comes out as tokens.
- A **forward pass** runs the input through all the model's layers once. Its output is one score for every token in the vocabulary (the *logits*).
- The model writes **one token per forward pass**. A 500-token answer needs 500 passes, one after the other.

This loop is called **autoregressive generation**. Everything in today's lecture is about making it faster or cheaper.

What a model is in memory

A model is mostly its **weights**: a long list of numbers.

$$\text{size in bytes} = \text{number of weights} \times \text{bytes per weight}$$

- **bf16** stores each number in 16 bits = 2 bytes. It is the normal precision for serving.
- **FP8** uses 8 bits = 1 byte, and **INT4** 4 bits: smaller, but slightly less exact.

Example: a model with 4 billion weights in bf16 needs $4 \times 2 = 8$ **GB**.

The GPU

The model is copied into the **GPU's own memory**, which is fast but limited. The GH200 GPUs on Roihu have 96 GB.

Two numbers describe a GPU: how fast it **computes**, and how fast it can **read its memory** (its *memory bandwidth*, about 4 TB per second on a GH200). Both will matter today.

Serving a model

A **server** keeps the model loaded on the GPU and answers requests over the network, from many users at the same time.

Programs called **inference engines** do this efficiently. Two common open-source ones are **SGLang**, which we use in Lab 3, and **vLLM**.

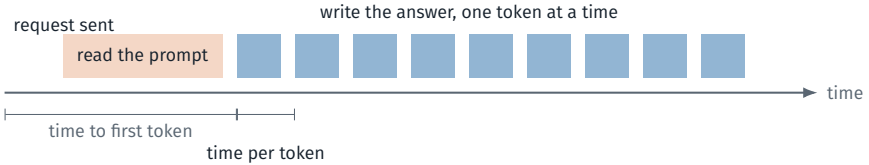
Most servers accept the same request format as OpenAI's API, so the same client code works with any of them.

A request

```
POST /v1/chat/completions
{"model": "task",
 "messages": [{"role": "user",
               "content": "Is this review
     positive?..."}],
 "temperature": 0,
 "max_tokens": 32,
 "stream": true}
```

The answer arrives piece by piece (*streaming*), so the user can start reading before the model has finished.

What we measure

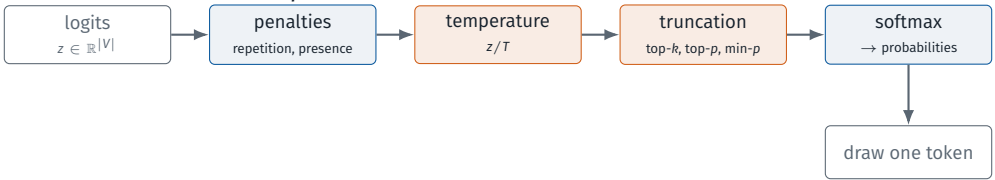


- **time to first token** (TTFT): how long until the answer starts
- **time per output token** (TPOT): how fast the answer then appears
- **throughput**: tokens per second for all users together, i.e. what the GPU produces
- **median** and **p95**: the typical request, and the slowest one in twenty
- **quality**: are the answers still as good? Always measured next to speed

Decoding

From logits to a token

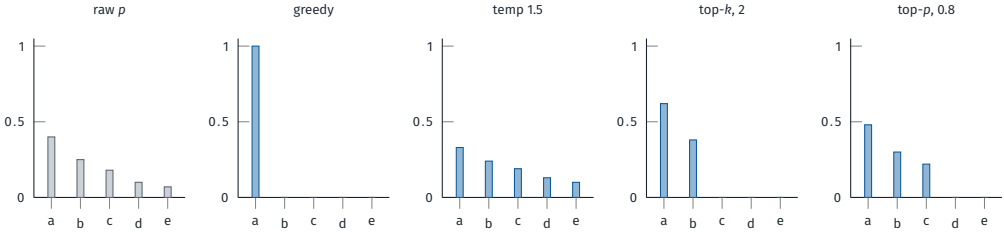
The forward pass gives one score (*logit*) for every token in the vocabulary, about 250,000 for Qwen3.5. A few short steps turn these scores into the one token that is written next.



Two things to notice:

- it is **cheap**: a few operations on one list of numbers, compared with billions of operations in the forward pass;
- it is **not part of the model**. The same weights behave very differently under different settings, which come from `generation_config.json`, your request, or the server's defaults.

Temperature and truncation change one distribution



greedy	take the most likely token.
temperature	values below 1 sharpen the distribution; values above 1 flatten it.
top-k	keep a fixed number of likely tokens, then renormalise.
top-p	keep the smallest set whose probabilities add up to p; the set adapts to confidence.

Which sampler for which task

Task	Sensible default	Why
Classification, extraction, exact-match QA	temperature 0 (greedy)	there is one right answer, so variance only costs you
Code, maths, anything verified	temperature 0 for one shot; about 0.8 if you sample several and vote	diversity only pays when you have a verifier to pick the winner
Open generation, dialogue, summarisation	temperature 0.7, with top-p 0.9 or min-p 0.05	greedy output is repetitive and flat, and a human is judging

Sample widely only when a human or an automatic checker can reject poor outputs. Otherwise sampling adds variance without a way to select the better result.

Source: Raschka, *Build Your Own Reasoning Model*, Oct 2026, on best-of-n and self-consistency.

Decoding parameters are part of your experimental setup

A sampling setting can come from three places:

1. your API call
2. the server's defaults
3. the model's own `generation_config.json`, shipped in the repository and silently applied by `transformers`

They can disagree: `transformers` uses the model's file, a server may or may not, and client libraries have their own defaults for `temperature` and `top_p`.

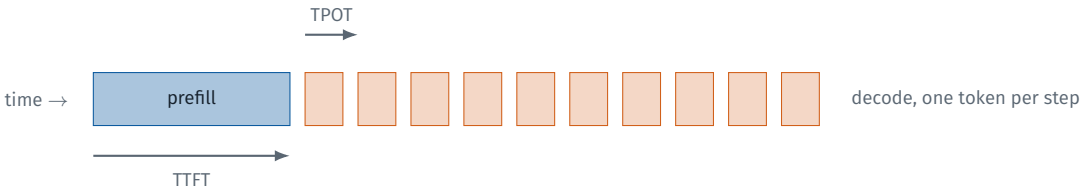
The rule for every experiment

Set decoding parameters **explicitly**, in one place, and record them next to every number you report. A speed-versus-quality table means nothing if the two rows used different temperatures.

Also fix `max_tokens` and the random seed, and report how much your measurements vary, not only the average.

Prefill vs decode

One request, two different computations



	Prefill	Decode
What	the whole prompt at once	one new token per step
Reads	every weight once, for 2,000 tokens	every weight once, for 1 token
Limited by	**how fast the chip multiplies**	**how fast memory feeds it**
Felt as	**TTFT** — wait for the first token	**TPOT** — gap between tokens

Fetching a number costs far more than multiplying it

A GPU does two things, and one is much more expensive than the other:

- **fetching** a weight from the card's memory: **slow**;
- **multiplying** with it once it has arrived: roughly a hundred times cheaper.

So speed depends on how much work you get out of each fetched weight.

Decode: one token at a time

For a 4B model, read all 8 GB of weights to produce one token. With 32 users in one batch, each fetched weight serves 32 tokens.

Prefill: the whole prompt at once

A 2,000-token prompt goes through in one pass, so each fetched weight is used 2,000 times.

Decode is limited by memory; batching several users is the main fix.

A BANDWIDTH ESTIMATE FOR DECODE SPEED

$$\text{tokens per second} \lesssim \frac{\text{memory bandwidth}}{\text{bytes of weights}}$$

That is: how many times per second can the card read the whole model?

Example: a 4B model in bf16 on one GH200

- 4 billion weights, 2 bytes each: **8 GB** to read for every single token
- the card moves **4.0 TB** of memory per second
- 8 GB goes into 4.0 TB about **500 times**, so 500 tokens a second, i.e. 2 ms each

That is the *upper limit* for one user. Real servers reach perhaps half of it.

Include the KV cache At 8k context, this example also reads about 1.2 GB of cached keys and values each step. The ceiling becomes

$$4.0\,\mathrm{TB/s} \div 9.2\,\mathrm{GB} \approx \textbf{430 tokens/s}.$$

Source: NVIDIA GH200 Grace Hopper Superchip datasheet (HBM3 and HBM3e variants); CSC, `docs.csc.fi/computing/systems-roihu`.

TTFT, TPOT, AND WHO FEELS WHICH

$$\text{total wait} = \underbrace{\text{TTFT}}_{\text{reading the prompt}} + \underbrace{\text{TPOT} \times (\text{tokens after the first})}_{\text{generating the answer}}$$

That is: you wait once for the prompt, then once per token produced.

Worked, 4B model, 2,000-token prompt, 500 output tokens:

- prefill: about 16 trillion operations, at the roughly 400 trillion per second the card really achieves: **40 ms**
- decode: 500 tokens at 5 ms each is **2.5 s**

Prefill is **1.6%** of the wall clock. Two thousand prompt tokens cost about as much time as **eight** generated ones.

Who feels what

TTFT — chat users, agents doing many short calls, anything with a spinner.

TPOT — anyone reading a streamed response. Anything under about 30 ms per token feels instant; 100 ms feels slow.

Neither — batch jobs. For offline scoring only *throughput* matters, so batch as much as memory allows.

Which technique helps which phase

Technique	helps prefill	helps decode
Bigger batches	no (already busy)	**yes, hugely**
INT4 weight-only quantisation	**can hurt**	**yes**
FP8 weights + activations	**yes**	**yes**
KV-cache quantisation	slightly	**yes**
Prefix caching	**yes, enormously**	no
Speculative decoding	no	**yes**

Take away

Work out which half of the request you are paying for before you optimise.

The KV cache

Why a cache?

In each layer, the model's *attention* lets the new token look back at all earlier tokens. For that, every earlier token is represented by two short lists of numbers, a **key** and a **value**.

These do not change once computed. So instead of recomputing them for the whole text at every step, the server **stores** them: this is the **KV cache**.

Without it, the 500th token would repeat the work for the 499 before it. With it, each step only computes the new token.

The price

The cache grows by one entry per token, in every layer, for every request that is running. It lives in GPU memory, next to the weights.

So the cache, not the weights, often decides how many users fit on one GPU. The next slides put numbers on this.

How big is the KV cache?

$$\text{KV bytes} = \text{requests} \times \text{tokens} \times \text{layers} \times \text{KV heads} \times \underbrace{\text{numbers per head}}_{128} \times \underbrace{2}_{\text{key and value}} \times \underbrace{2}_{\text{bytes each, bf16}}$$

That is: every layer stores two small vectors (a key and a value) for every token of every request. A typical 4B model (Qwen3-4B) has 36 layers and 8 KV heads, so

$$36 \times 8 \times 128 \times 2 \times 2 \;\approx\; 147\,\mathrm{kB} \text{ per token}$$

Why some models need less

Many models share one key and value between several attention *heads* (fewer "KV heads"), or look back only over a window of recent tokens. Both make the cache smaller.

Every decode step also *reads the whole cache*, so it costs speed as well as memory. Qwen3.5 keeps a KV cache in only one layer in four (the others use linear attention), so its cache grows about four times more slowly.

The same model at 8k and at 128k

Context	KV / request	× weights (8 GB)	of 96 GiB
1 request @ 8k	1.2 GB	0.15×	1%
1 request @ 32k	4.8 GB	0.6×	5%
1 request @ 128k	19.3 GB	2.4×	19%

One user at 128k context occupies **more than twice the memory of the entire model**.

Double the context and you double the cache, which halves the number of users the GPU can hold.

Units, stated once

96 GiB = 103 GB.
A server typically reserves ≈ 90% of the card, and the weights take 8 GB, so

$$0.9 \times 103 - 8 \;\approx\; \mathbf{85}\,\mathrm{GB}$$

is what is actually available for KV. We use 85 GB from here on.

Multiply by concurrency until it breaks

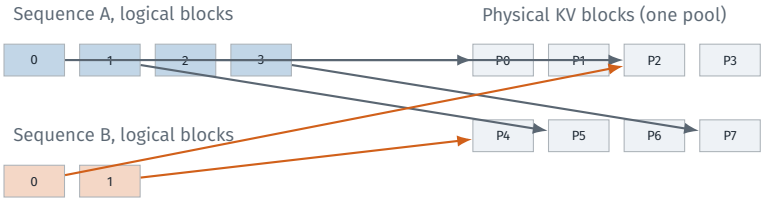
Concurrent	KV @ 8k	+ 8 GB weights	
1	1.2 GB	9.2 GB	trivially fine
8	9.6 GB	17.6 GB	fine
32	38.6 GB	46.6 GB	fine
64	77 GB	85 GB	at the limit
70	84 GB	92 GB	over budget

At **128k** the same arithmetic gives **4** concurrent requests: sixteen times the context means sixteen times fewer users. The GPU could still compute more, but has no memory left.

The conclusion of this section

In this example, the KV cache sets the concurrency limit. The 4B model's weights take 8 GB, while its cache can grow to many times that at long contexts and high concurrency.

PagedAttention



- the cache is stored in small fixed-size **blocks**, handed out only when a request needs more
- a table per request records where its blocks are, so they need not be next to each other
- without this, servers reserved the maximum length for every request and wasted 60–80% of the cache memory; with it, **under 4%**
- requests that start with the same tokens can **share** blocks

Source: Kwon et al., SOSP 2023 (vLLM). SGLang keeps its cache in a similar shared pool.

Prefix caching: paying for the system prompt once

If a prompt **begins with** tokens whose KV blocks are still cached, the server reuses them and skips that part of prefill.

SGLang does this automatically (it calls it *RadixAttention*). The answers do not change; it only saves work, as long as the shared prefix is still in the cache.

In Lab 3 it is switched off for the speed test (`--disable-radix-cache`), so repeated prompt starts do not make the server look faster than in normal use.

Worked: a 5-shot evaluation run

Shared prefix: system prompt + 5 examples ≈ 1,200 tokens.
Per-item suffix: ≈ 80 tokens.

Without prefix caching you prefill 1,280 tokens per item. With it, **80**.

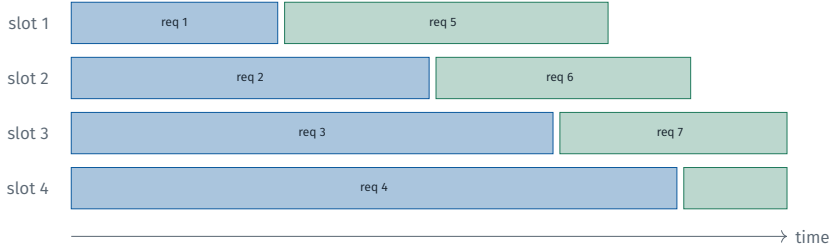
Over a 1,000-item test set that is up to 16× **less prefill work** while that prefix remains cached.

The same idea helps chat, agents and RAG, which repeat long instructions.

Source: Zheng et al., *SGLang: Efficient Execution of Structured Language Model Programs*, 2024 (RadixAttention); vLLM automatic prefix caching.

Batching and the server

Continuous (iteration-level) batching



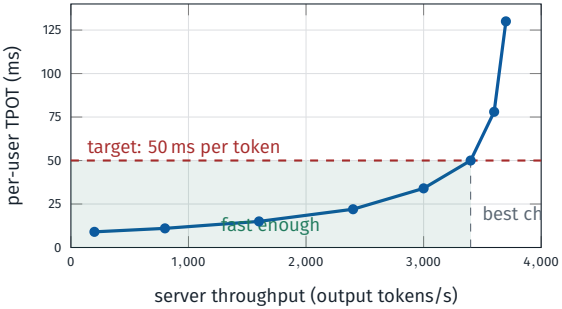
Older servers started a batch and waited for its slowest request before taking new ones. Now the server rebuilds the batch **at every step**:

- a request that has finished leaves at once and frees its memory
- a waiting request joins at the next step
- no slot sits idle while others finish

This gives roughly **2–4 times** more throughput at the same latency.

Source: Yu et al., OSDI 2022; Kwon et al., SOSP 2023.

Throughput and latency pull in opposite directions



More parallel requests give more tokens per second in total, but each user waits longer per token. Past the bend, extra requests only slow everyone down. Choose the point where the speed per user is still acceptable: that is what Lab 3 measures at 1, 8 and 32 parallel requests.

Source: Shape after published vLLM and DistServe throughput–latency sweeps; values illustrative.

The flags that matter

```
# SGLang, as in Lab 3 (written on one line in practice)
python -m sglang.launch_server --model-path Qwen/Qwen3.5-0.8B
  --context-length 4096       # longest prompt + answer
  --max-running-requests 64   # how many requests run at once
  --mem-fraction-static 0.8   # GPU memory for weights + KV cache
  --quantization fp8          # FP8 weights (leave out for bf16)
  --disable-radix-cache       # prefix caching off: speed tests only
  --port 30000
```

`context-length`	the biggest lever on how many users fit; do not keep a 128k default if your task needs 2k
`max-running-requests`	raising it past what the KV memory allows only makes requests wait inside the server
`mem-fraction-static`	how much of the GPU the server takes; lower it if you run out of memory
`disable-radix-cache`	prefix caching is on by default; keep it on in real use

Quantisation

Three independent places to use fewer bits

What	Size, 4B model	What lowering it buys
Weights	8 GB in bf16	fewer bytes read per decode step ⇒ directly faster decode
Activations (numbers computed inside the pass)	small, temporary	lets the GPU multiply in FP8 ⇒ faster *prefill*
KV cache	1.2 GB per 8k request	more concurrent users, and less to re-read every step

These are **independent choices**. "The model is quantised" says little: always ask *which of the three* uses fewer bits, and in what format. Lab 3 tests FP8 weights.

FP8 on a GH200

What it is

- weights (and often activations) use 8-bit numbers instead of 16-bit;
- each block of numbers also stores a scale factor, because 8 bits cover a much smaller range;
- the GH200 can multiply FP8 numbers directly, without unpacking them first.

Why it is used

- weights halve: 8 GB $\rightarrow$ 4 GB;
- less weight traffic speeds up decode;
- the GPU multiplies FP8 faster, which can also speed up prefill.

How to evaluate it Run the same test set and scorer, and report quality next to speed. Small average losses in papers do not guarantee that your language or task is unaffected.

Source: NVIDIA Hopper FP8 documentation; SGLang quantization documentation.

INT4 WEIGHT-ONLY: AWQ AND GPTQ

Store each weight in 4 bits and **unpack it to 16 bits just before use**. Only the stored weights shrink.

GPTQ rounds each layer's weights so that the layer's *output* changes as little as possible on sample data.

AWQ protects the small share of weights that matter most for the outputs.

Sources: Frantar et al., *GPTQ*, ICLR 2023; Lin et al., *AWQ*, MLSys 2024.

Why weight-only works at all

Decode waits for memory. Weights of 2 GB instead of 8 GB mean a quarter of the reading per token, so the upper limit goes from 500 to **2,000 tokens/s**. Prefill gains little, and the unpacking can even slow it down.

Both need **calibration data**: a few hundred example texts. Use data like your task; English web text is a poor choice for a Finnish model.

The quality cost, and where it shows up

For FP8 and good INT4 the published quality loss is usually **under a point** on MMLU-style benchmarks. That average hides where the damage actually lands:

- **long-tail inputs**: rare formats, unusual prompts
- **non-English text**, especially morphologically rich languages
- **long reasoning chains**, where one bad token derails the rest
- **long contexts**, if the KV cache is quantised

So Lab 3 scores it

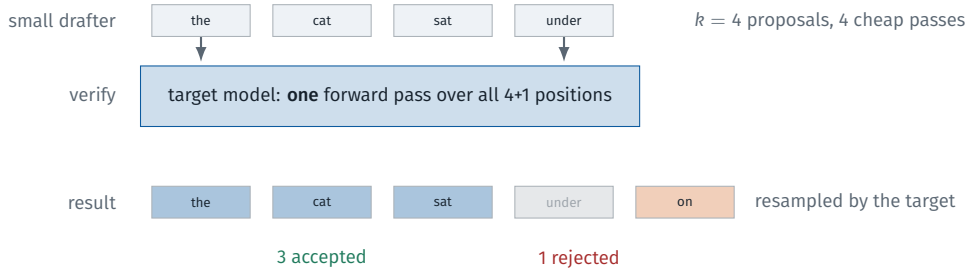
Serve the model in FP8, run the same test set and scorer as in Lab 2, and put the score in the same table as the tokens per second. **Always report the quality number next to the speed number.**

Take away

FP8 is a good default on a GH200; INT4 mainly speeds up decode. Either way, trust the quality number you measured on your own task.

Speculative and constrained decoding

Speculative decoding



A small, fast model guesses the next few tokens; the big model checks all the guesses in **one** pass, which costs about the same as producing one token. Here one pass gives four tokens. The guesses are accepted in a way that keeps the output exactly as if the big model had produced it alone.

Sources: Leviathan et al., *Fast Inference from Transformers via Speculative Decoding*, ICML 2023; Chen et al., *Accelerating LLM Decoding with Speculative Sampling*, 2023.

Constrained decoding

What it is

- Give the server a schema or grammar for the required output.
- At each step, mask tokens that would make the output invalid.
- Sample from the tokens that remain.

Why it is used

Applications often need parseable JSON, and retries add latency and cost.

What it guarantees

Valid syntax when generation completes successfully. The constraint removes invalid token sequences, but truncation and server failures still need handling.

What it does not guarantee

Anything about the content. The label inside valid JSON can still be wrong, and forcing the format can even make it worse.

In SGLang you send a JSON schema with the request (`response_format`); libraries such as **XGrammar** make the masking fast. Lab 3 measures this.

Sources: Willard & Louf, *Efficient Guided Generation for LLMs*, 2023 (Outlines); Dong et al., *XGrammar*, 2024.

Valid syntax can still reduce task accuracy

A schema guarantees the *shape* of an output, not its content. If the model would naturally write something else first, forcing the format can push it towards a worse answer.

	invalid output	task score
free generation + parser	varies	baseline
JSON-schema constrained	0%	measure it

Report **both** invalid-output rate and task score. For reasoning models, let the model reason freely and constrain only the final answer.

Cost of serving

Cost per million tokens, from first principles

$$\text{EUR per million tokens} = \frac{\text{EUR per GPU-hour}}{\text{tokens you got out of that hour}} \times 10^6$$

That is: you rent the GPU by the hour, so divide the rent by what the hour produced. In this example the rent is EUR 2.50/h and the GPU is busy only 40% of the time, so the result is divided by 0.4.

32 users at once, measured 2,500 tokens/s:

- $2500 \times 3600 = 9$ million tokens an hour
- EUR 2.50 for 9 million = EUR 0.28 per million
- busy only 40% of the time: **EUR 0.69 per million**

One user, measured 120 tokens/s: 0.43 million tokens an hour, the same arithmetic, **EUR 14.5 per million**.

The same GPU, the same model, 21× the price

Only the number of parallel requests changed. Batching decides most of the cost per token.

Public prices in late 2026: roughly \$0.02–0.10 per million output tokens for hosted 4B-class open models, \$5–15 for frontier ones.

Source: GPU price: representative on-demand H100/GH200 cloud rate, late 2026. Throughput: the kind of figure Lab 3 produces. API prices: public price lists, November 2026; they change quickly.

Three things to remember

1. **The two halves of a request hit different limits.** Reading the prompt (prefill) is limited by computation; writing the answer (decode) is limited by memory. Find out which one you are paying for before you optimise.

2. **The KV cache often sets the concurrency limit.** In the worked example the weights take 8 GB and the cache can take the rest of the GPU. Paging, prefix caching and quantisation all save cache memory.

3. **Measure latency and quality together.** Warm up the server, measure TTFT and TPOT separately, try several numbers of parallel requests, and report the median and p95. A faster setting is only useful if the answers stay good enough.

Lab 3: you serve a model with SGLang, measure it in bf16 and FP8, and compare the numbers with today's estimates.

A server configuration you can reuse

If the same endpoint suits your project, it can become **infrastructure for later labs**:

- **Week 5, RAG**: your server generates the answers from the retrieved text.
- **Week 6, agents**: the same server answers many short calls, so TTFT and prefix caching matter.
- **Week 7**: your demo runs against it.

The file worth keeping

Lab 3 keeps all server settings in `configs/bf16.json`, and `bash scripts/start_server.sh configs/bf16.json` starts the same server again in later weeks.

The steps are in `Week-3/Lab-3/README.md`.

A running server keeps its GPU memory whether or not anyone uses it. Stop it when you are done.

Reading

Serving systems

- Zheng et al. 2024, *SGLang* / RadixAttention
- Kwon et al. 2023, *PagedAttention* (vLLM)
- Yu et al. 2022, *Orca*, continuous batching
- Agrawal et al. 2024, *Sarathi-Serve*
- Zhong et al. 2024, *DistServe*
- Pope et al. 2023, *Efficiently Scaling Transformer Inference*

Kernels

- Dao et al. 2022, Dao 2023, *FlashAttention* 1 and 2
- Williams et al. 2009, the roofline model

Decoding

- Leviathan et al. 2023, speculative decoding
- Chen et al. 2023, speculative sampling
- Cai et al. 2024, *Medusa*; Li et al. 2024, *EAGLE*
- Willard & Louf 2023, *Outlines*; Dong et al. 2024, *XGrammar*

Quantisation

- Frantar et al. 2023, *GPTQ*; Lin et al. 2024, *AWQ*
- Hooper et al. 2024, *KVQuant*

Further reading

- Thinking Machines Lab 2025, *Defeating Nondeterminism in LLM Inference*